

UNITAT 4 · SETMANES 5-6

# Processos i memòria

📅 Setmanes 5-6 · dates per confirmar </> Laboratori TD3 (resolt + repte IA) ~ 7 animacions

MANUAL M4 · TEORIA DE LA UNITAT

## Processos i memòria

Lectura bàsica de la unitat 4 ( $\approx 2$  h, dues setmanes). Cada secció acaba amb un enllaç **Practica-ho** →. Les sortides són reals: programes del TD3 executats a la màquina del curs (Ubuntu 26.04, Linux 7.0, x86-64).

### CONTINGUT

- |                                                       |                                                                   |
|-------------------------------------------------------|-------------------------------------------------------------------|
| §1 El procés i el seu espai d'adreces                 | §4 <code>fork</code> , còpia en escriptura i memòries intermèdies |
| §2 Memòria virtual: pàgines, fallades i substitució   | §5 <code>exec</code> , <code>exit</code> i <code>wait</code>      |
| §3 El procés per al nucli: PCB, estats i planificació | §6 El primer procés, les sessions i els serveis                   |

### §1 El procés i el seu espai d'adreces

Un **procés** és un programa en execució juntament amb el seu context: memòria, registres, descriptors oberts, credencials. Cada procés té un **espai d'adreces privat**: les adreces que fa servir són *virtuals*, i la mateixa adreça en dos processos apunta a memòria física diferent. A x86-64 els punters tenen 64 bits però se'n fan servir 48: 256 TiB, la meitat baixa per al procés i l'alta per al nucli.

L'espai s'organitza en **zones**: el *codi* (text, només de lectura i executable), les *dades* (globals inicialitzades) i el *bss* (globals sense inicialitzar), el *heap* (`malloc` petits), les regions *mmap* (biblioteques i `malloc` grans) i la *pila* (variables locals, que creix cap avall). El programa **zones** del TD3 imprimeix una adreça de cada zona i la regió de `/proc/self/maps` que la conté:

### Sortida real

```
$ ./zones
zone                address    perm  region
code   (main)        0x5f935f9d1350  r-xp  ~/fso/tds/bin/td3/zones
data   (initialized)   0x5f935f9d5010  rw-p  ~/fso/tds/bin/td3/zones
bss    (global_array)   0x5f935f9d5040  rw-p  ~/fso/tds/bin/td3/zones
heap   (malloc 10 B)     0x5f938dc96010  rw-p  [heap]
heap   (end: sbrk(0))  0x5f938dcb6fff  rw-p  [heap]
mmap   (malloc 64 MiB) 0x71b340fff010  rw-p  (anonymous)
libc   (printf)         0x71b345064080  r-xp  /usr/lib/x86_64-linux-gnu/libc.so.6
stack  (local)            0x7fff6c1e68b4  rw-p  [stack]
```

Si l'executes diverses vegades, les adreces canvien: el nucli fa aleatòria la base de cada regió (**ASLR**, *address space layout randomization*) perquè un atacant no pugui predir on són el codi o la pila. glibc serveix els `malloc` de 128 KiB o més amb un `mmap` propi, fora del heap, que es retorna al sistema tan bon punt s'allibera.

### Practica-ho → TD3 · 5 ( zones )

## §2 Memòria virtual: pàgines, fallades i substitució

La memòria es divideix en **pàgines** (4 KiB a x86-64; també pàgines grans de 2 MiB i 1 GiB) i la memòria física en **marcs** de la mateixa mida. Una **taula de pàgines** per procés diu en quin marc és cada pàgina i amb quins permisos (lectura, escriptura, execució, usuari). A x86-64 la taula té 4 nivells: una adreça virtual es divideix en quatre índexs de 9 bits i un desplaçament de 12 bits. El maquinari —la **MMU**— tradueix a cada accés, i la **TLB** guarda a la memòria cau les traduccions recents per no recórrer la taula cada vegada.

Si l'entrada de la taula no hi és, la MMU llança una **fallada de pàgina** i salta al nucli. Si l'adreça pertany a una regió vàlida, el nucli assigna un marc, el posa a zero (o el llegeix del disc) i torna a executar la instrucció: això és l'**assignació mandrosa**. Si no pertany a cap regió, o l'accés viola els permisos (escriure al codi, executar a la pila), envia `SIGSEGV`. Mesurat amb `lazy`:

### Sortida real

```
$ ./lazy 1024
start                VmRSS =      1756 KiB    page faults so far = 0
malloc(1024 MiB) returned 0x7fadb67ff010
after malloc          VmRSS =      1952 KiB    page faults so far = 8
after touching half of the pages  VmRSS =    526300 KiB    page faults so far = 131081
pages touched: 131072 (4096 bytes each)  page faults: 131073    1968 ns per fault
after free            VmRSS =      2012 KiB    page faults so far = 131081
```

Linux també és *optimista* (overcommit): promet més memòria de la que té, perquè gairebé ningú no fa servir tot el que reserva. Si de debò s'acaba, l'**OOM killer** acaba processos. Quan no

queden marcs lliures, el nucli tria una pàgina **víctima** i l'envia al disc (*intercanvi*) o la comprimeix a la RAM (zswap, zram). L'algorisme ideal (OPT) trauria la pàgina que s'usarà més tard; els reals s'aproximen a LRU amb el bit d'accés que posa la MMU (l'algorisme del rellotge; a Linux, llistes activa/inactiva i MGLRU). Amb FIFO pot passar l'**anomalia de Belady**: més marcs, més fallades. Si el conjunt de pàgines en ús no cap a la memòria, el sistema passa més temps paginant que executant: **thrashing**.

Practica-ho → A4.1 memòria virtual · A4.5 substitució de pàgines

### §3 El procés per al nucli: PCB, estats i planificació

El nucli descriu cada procés amb un **bloc de control** (PCB; a Linux, `struct task_struct`): PID i PPID, propietari i permisos, estat, registres desats, prioritat, temps de CPU, taula de descriptors, límits (`getrlimit`), senyals i taula de pàgines. Es pot inspeccionar a `/proc/PID/`, que és el que llegeixen `ps` i `top`.

Un procés passa per diversos **estats**: *preparat* (esperant la CPU) i *en execució* —tots dos `R` a `ps`—, *bloquejat* esperant E/S (`S`, o `D` si no es pot interrompre), *aturat* per `SIGSTOP` / `SIGTSTP` (`T`) i *zombi* (`Z`) entre el seu final i el `wait` del pare.

Passar la CPU d'un procés a un altre és un **canvi de context**: desar els registres d'un, carregar els de l'altre i canviar de taula de pàgines. El **planificador** decideix a qui li toca. Els algorismes clàssics: FCFS (ordre d'arribada; pateix l'efecte comboi), SJF i SRTF (el més curt primer; minimitzen l'espera mitjana però exigeixen estimar durades), Round Robin (torns d'un *quàntum*; bona resposta, més canvis de context) i prioritats (amb risc d'inanició, que es corregeix amb l'envelliment). Des de la versió 6.6 (2023) Linux fa servir **EEVDF**, que reparteix la CPU per temps virtual ponderat per la prioritat (`nice`) i serveix primer el termini més proper.

Practica-ho → A4.3 estats · A4.4 planificació

### §4 `fork`, còpia en escriptura i memòries intermèdies

A Unix un procés nou neix com a **clon** d'un altre: `fork()` retorna 0 al fill, el PID del fill al pare i -1 si falla. El fill rep una còpia de tot: memòria, descriptors (que apunten a les mateixes entrades de la taula de fitxers oberts, unitat 3), gestors i màscara de senyals —però no els senyals pendants—. Copiar tota la memòria seria molt car, així que Linux fa servir la **còpia en escriptura**: només copia les taules de pàgines, ho marca tot com a només de lectura i copia cada pàgina només quan un dels dos hi escriu:

### Sortida real

```
$ ./cow 256
parent: 256 MiB filled (65536 pages of 4096 bytes)
fork took 6.70 ms (only the page tables are copied)
child reads 65536 pages: 0 page faults (shared, nothing copied; sum 7864320)
child writes 16384 pages: 16384 page faults (one copy per page), 2753 ns per copy
parent: the child exited with status 0; my copy still says 'x'
```

La còpia també inclou el que la biblioteca de C té pendent. `printf` no crida `write`: acumula en una **memòria intermèdia** que es buida per línia (`stdout` cap a un terminal), quan s'omple (`stdout` cap a una canonada o un fitxer), amb `fflush` o en l'`exit`; `stderr` no té memòria intermèdia i `_exit` no buida res. Si encara hi havia text a la memòria intermèdia en el `fork`, pare i fill l'escriuen tots dos: apareix dues vegades.

### Sortida real

```
$ ./fork_pid_0 | cat
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

## Practica-ho → A4.2 còpia en escriptura · A4.6 memòries intermèdies · TD3 · 2 i 3

### §5 `exec`, `exit` i `wait`

`execve(path, argv, envp)` substitueix el programa del procés: carrega codi i dades nous, crea un heap i una pila nous i salta al punt d'entrada. Conserva el PID, el PPID i els descriptors oberts (excepte els marcats amb `0_CLOEXEC`); amb el bit SUID, l'UID efectiu passa a ser el del propietari del fitxer. Si funciona, no torna. Les variants de biblioteca (`execl`, `execv`, `execvp`...) canvien com es passen els arguments i si es busca al `PATH`. Entre `fork` i `exec` el fill pot reorganitzar els seus descriptors: així implementa el shell `>`, `<` i les canonades (el `launch` del TD3 · 4, i la unitat 5).

Un procés acaba amb `exit(status)` —que crida les funcions `atexit`, buida les memòries intermèdies de `stdio` i crida la crida al sistema `_exit` — o per un senyal. El nucli tanca els seus fitxers i n'allibera la memòria, passa els fills vius a `init` o al *subreaper* més proper i avisa el pare amb `SIGCHLD`. El PCB queda com a **zombi** fins que el pare crida `wait` o `waitpid`, que retorna l'estat empaquetat; les macros el descodifiquen: `WIFEXITED` / `WEXITSTATUS`, `WIFSIGNALED` / `WTERMSIG`, `WIFSTOPPED` i `WIFCONTINUED` (amb les opcions `WUNTRACED` i `WCONTINUED`; `WNOHANG` no bloqueja).

### Sortida real

```
$ ./lifecycle
1) the child 356667 has exited, but nobody has called wait yet:
    PID    PPID STAT COMMAND
    356667 356666 Z    lifecycle
waitpid → WIFEXITED = 1, WEXITSTATUS = 7; now ps finds nothing:
    PID    PPID STAT COMMAND
2) orphan: my PID is 356666
   grandchild 356679: parent was 9771, now it is 9771 (systemd)
```

### Sortida real

```
$ ./launch out.txt ./segfault
launch: ./segfault killed by signal 11 (SIGSEGV), core dumped
[exit status 139]
```

## Practica-ho → A4.7 fork, exec i wait · TD3 · 4

### §6 El primer procés, les sessions i els serveis

Si cada procés neix d'un `fork`, algú ha de ser el primer. El **procés 0** el construeix el nucli en arrencar (un per CPU, en mode nucli): inicialitza la memòria virtual, crea els fils del nucli (`kthreadd`, PID 2) i el **procés 1**, que passa a mode usuari i executa `/sbin/init` —avui **systemd** a gairebé totes les distribucions—, inicia els serveis i adopta els orfes. A la màquina del curs, `ps -p 1 -o comm=` respon `systemd` (TD0).

Una **sessió** agrupa processos, normalment els d'un terminal, amb un líder; si el líder acaba, els altres reben `SIGHUP`. `setsid()` crea una sessió nova sense terminal de control. Un **dimoni** clàssic es desenganxava del terminal amb `fork`: el pare sortia i el fill cridava `setsid`. Avui, a Linux, els serveis llança `systemd`: el programa s'executa en primer pla i `systemd` s'ocupa de la sessió, els registres (`journalctl`), els reinicis i els límits de recursos (`cgroups`, unitat 9).